* Discussed 6 with Darren/Chris/TA at Office hours. Did not use much in final solution.

HW 1 [redacted] due 10/14

1)a) Base case: $n=1$

$3(1)-2 \stackrel{?}{=} \frac{(1)(3(1)-1)}{2}$

$1 \stackrel{\checkmark}{=} 1$

Induction step: Assume the statement $1+4+7+\cdots+(3n-2) = \frac{n(3n-1)}{2}$ is true.

let $n = k+1$

$\underbrace{1+4+7+\cdots+(3k-2)}_{\uparrow} + (3(k+1)-2) \stackrel{?}{=} \frac{(k+1)(3(k+1)-1)}{2}$

substitute using assumption

$\frac{k(3k-1)}{2} + (3k+1) \stackrel{?}{=} \frac{(k+1)(3k+2)}{2}$

$k(3k-1) + 6k + 2 \stackrel{?}{=} (3k^2+5k+2)$

$3k^2 - k + 6k + 2 = 3k^2+5k+2$

$3k^2+5k+2 = 3k^2+5k+2$ ✓

b) Assume for the sake of contradiction that x and y are integers, and $x^2 - y^2 = 1$.

Factor and solve. ① $(x+y)(x-y) = 1$

② $x+y=1$ and $x-y=1$

$2x = 2$

$x = 1$

③ $x+y=1$

$1+y=1$

$y=0$

④ $x+y=-1$ and $x-y=-1$

$2x = -2$

$x = -1$

⑤ $-1+y=1$

$y=0$

x is not a positive integer, neither is y.

This is a contradiction because y cannot be both 0 and a positive integer. x and y cannot be positive and integers that solve $x^2-y^2=1$ as written in the problem.

2)

A	B	slower: A ∈ O(B)	as fast (at most): A ∈ O(B)	same: A ∈ Θ(B)	as fast (at least): A ∈ Ω(B)	faster: A ∈ W(B)
$\log_2(n)$	$\log_5(n)$	NO	YES	YES	YES	NO
$\log\log(n)$	$\sqrt{\log(n)}$	YES	YES	NO	NO	NO
$n^{4/5}\log(n)$	$n^{\log(n)}$	YES	YES	NO	NO	NO
$2^{\log_6(n)}$	n^6	YES	YES	NO	NO	NO
$n^3 2^n$	3^n	YES	YES	NO	NO	NO

3) a) Outer for loop iterates n times (from 1 to n). Inner loop increments by 2 each time, so would run $\frac{i}{2}$ times, which simplifies to $i \to n$ (since i goes till n in outer loop). Multiplying gives the time complexity $\Theta(n^2)$.

b) The while loop would run n times, but we are incrementing i to i^2 each iteration.

i	i^2	K
2	4	1
4	16	2
16	256	3

At each K iteration, i is 2^{2^K}. termination: $2^{2^K} \leq n$ (since that is the loop condition)

$\log_2 2^{2^K} \leq \log_2 n$

$2^K \leq \log_2 n$

$\log 2^K \leq \log\log n$

$K \leq \log\log n$.

There for the loop keeps going while K is $\leq \log\log n$, or for that many iterations, giving time complexity $\Theta(\log(\log(n)))$.

4. Algorithm:

1. Initialize an empty dictionary to store the elements and frequencies of each number.
2. For loop through to the end of array (use the array size given as input)
    a. If the element is not in the dictionary, add it with frequency 1.
    b. If the element already exists, increment its frequency by 1.
3. Initialize a min_element and min_frequency (as infinity)
4. For loop through dictionary
    a. If (the current value is less than the min_frequency) or (the value is equal to the min_frequency and the key is less than the min_element)
        i. Set min_frequency as the value
        ii. Set min_element as the key
5. Return min_element

Time Complexity Analysis: O(n)

- Step 1 is constant time, O(1).
- Step 2 has time complexity O(n), since you are looping through n elements.
- Step 3 is constant time, O(1).
- Step 4 also has time complexity O(n), the dictionary can have at most n key, value pairs, since every element in the array could be unique. This means we are looping through n pairs of the dictionary.
- Step 5 is O(1).
- Overall: 1+n+1+n+1 simplifies to O(n).

Proof of Correctness: Proof by Induction

Base case: n = 1

- For this base case, the algorithm's first loop will result in a dictionary with one pair, where the key is the element and the value is 1 (since its frequency is 1). In the second loop, min_frequency will be set to 1 (since it is less than inf and the first condition is met) and min_element will be the only element in the array. The algorithm will return the element. This is the correct result and the base case holds true.

Inductive Step:

- Assume that the algorithm returns the correct minimum element with the smallest frequency for any array of size n, where n is greater than or equal to 1. Consider the input array with size n + 1. Based on the base step and assumption, the algorithm works correctly for n elements. For the n + 1th element, the algorithm will add its frequency and value to the dictionary. Then, if its frequency is less than the min_frequency from the last n elements, or if it is the same but the element is smaller than min_element, it will correctly update min_frequency and min_element to correspond to the n+1th element.

Termination:

- Algorithm will terminate since the first loop would run with the condition of the input size n (aka each iteration runs once for each element in the array).
- The second loop will run once for each key in the dictionary, with the condition as the length of the dictionary.

- Return at the end.
- Overall: Since both loops have ending conditions, the algorithm will terminate. At most, there will be time complexity O(n).

5. Algorithm:

1. Initialize an empty dictionary, P, which will have all the current partners of students.
2. Each hospital and student has a preference list.
3. Each hospital has a capacity, which is the number of students it will accept, and a list of tentatively accepted students.
4. While (some hospital h has any spots left and has not proposed to every student):
    a. Let s be the first student on h's preference list that h has not yet proposed to.
    b. If (s is unmatched), add s to h's accepted students list. Also, add the pair to P.
    c. Else
        i. Let h' be the current partner of s.
        ii. If (s prefers h to h')
            1. Remove s from h' 's accepted list and add s to h's accepted list.
            2. Remove the old pair from P.
            3. Add the new pair to P.
        iii. Else
            1. s rejects h
5. Return P.

Time Complexity Analysis: O(m*n)

- Initialization of dictionaries is O(1) time. For the while loop, in the worst case each hospital can propose to at most each student on its preference list once. There are m hospitals and n students. This loop could run at most O(m*n) times. Every step in the while loop is O(1), as well as returning P.

Correctness Proof:

Assume for the sake of contradiction that this algorithm does not result in a stable assignment, and there exists some instability.

Instability 1:

Case 1: h never proposes to s'

1. The first type of instability, as described in the problem, would mean that h prefers s' to s, its current partner.
2. One way it can be true that h prefers s' to s but ended up with s if it never proposed to s'.
3. But, if h is matched to s, h proposed to s.
4. And if h prefers s' to s, and proposes in decreasing order of preference, he must have also proposed to s' – this is a contradiction to our claim that h never proposed to s'.

Case 2: h proposed to s'

1. In the case that h proposed to s' and prefers s' to s, they would be paired.

2. Since s' is unmatched, it cannot reject h–unmatched students always accept proposals.
3. So if h preferred s' to s and proposed to s', it would stay with it over s.
4. This is a contradiction to the claim that s' is unmatched.

Instability 2:

For the second instability, as described in the problem, h could have never proposed to s' (which would lead us to the same contradiction as above), or h could have proposed to s'.

Case 1: h never proposes to s'

1. See same proof as above case 1.

Case 2: h proposed to s'

1. Since s' is not matched with h at the end of the algorithm, if the proposal happened, s' rejected h in favor of another hospital h'' that s' prefers to h.
2. This is because students only reject hospitals if the current one they are matched with is more preferable (sequence of hospitals they match with gets better and better).
3. s' must prefer their final partner h' over h'' and h.
4. This is a contradiction – s' would not prefer h to h'. If it did and h proposed, then s would not be matched with h'.

Termination: While loop terminates once all hospital's lists are at capacity. This happens after at most n*m iterations of the while loop, since each m hospital can propose to at most every n student.

Completeness: Since the algorithm only terminates when all hospitals are at full capacity, we fulfill the completeness aspect of the solution given in the problem that each hospital should fill all available positions.

Show that there is always a stable assignment of students to hospitals: As shown by my correctness proof (no instabilities, termination, and completeness), there is always a stable assignment of students to hospitals.

6. Algorithm:

1. Initialize an empty set, M, for matchings.
2. Each ship has a schedule, or preference list, that says for each day of the month which port it is currently visiting or if it is out at sea.
3. Initialize a dictionary D that will store all the ports preference lists as values.
4. Create preference lists for the ports based on the ship's schedules.
    a. For each port p: initialize an empty list in D.
    b. For each day in the month (until m-1 is reached):
        i. For each ship s (until n-1 is reached):
            1. If a port p is encountered, add s to p's preference list.
    c. For each list in D

        i. Reverse the list, since ports prefer ships that visit it last so it can be open as long as possible for other ships.
5. While (any ship s is unmatched and has not yet proposed to every port)
    a. Let p be the first port on s's list that s has not yet proposed to.
    b. If (p is unmatched)
        i. Add (s,p) to M.
    c. Else
        i. Let s' be the current partner of p.
        ii. If (p prefers s to s')
            1. Remove (s',p) from M.
            2. Add (s,p) to M.
        iii. Else
            1. p rejects s.
6. Output M. The port that ships are matched with is where their schedule gets truncated.

Time complexity analysis: O(mn)

1. Steps 1-3 are initialization, and have time complexity O(1).
2. For creating the preference lists (step 4):
    a. Initializing a list for each port is O(n) since there are n ports.
    b. Looping through each day of each ship's schedule is in worst case O(m*n), since there are m days and n ships.
    c. Looping through each list in D is O(n), since there are n ports and we reverse each port's list – O(n), giving us $O(n^2)$.
3. The while loop in step 5 can run at most n^2 times, since there are n ships and n ports they can propose to.
4. This results in $O(1) + O(n) + O(mn) + O(n^2) + O(n^2)$. Since m is greater than n, the dominant term is O(mn).

Correctness proof: I adapted this algorithm after the Gale-Shapley algorithm, so we know that when there are complete preference lists and we follow the same algorithm - stable matching can always be found (instability proof mentioned in problem 5).

In this context, the only difference is that I created preference lists for the ports. However, I made the lists so ports prefer the ship that visits it LAST–indicating that other ships that have to visit it to get to their assigned maintenance port have already done so. Therefore, there will never be 2 ships that will be at the same port at the same time.

Since a ship s can "kick out" other ships (say s') that will then find another port later, this also ensures that no two ships are ever assigned to the same port. This is similar to students switching hospitals if they prefer another more in the G-S algorithm.

Finally, we will always find a set of truncations since I ensure termination. The for loops will terminate since they only iterate over a finite number of ships or ports. For the while loop, we

can ensure the entire algorithm will terminate after at most n^2 iterations of the loop, as described in the time complexity analysis.

7. Let there be three men (1-3) and three women (4-6), with the following preference lists (from highest to lowest):

1: 4, 5, 6
2: 5, 4, 6
3: 4, 5, 6

4: 2, 1, 3
5: 1, 2, 3
6: 1, 2, 3

In this situation, this would be the final result (when men are proposing to women):
1-4
2-5
3-6

Suppose w lies about her preferences and says her list is: 2, 3, 1

This makes the new preference lists:
1: 4, 5, 6
2: 5, 4, 6
3: 4, 5, 6

4: 2, 3, 1
5: 1, 2, 3
6: 1, 2, 3

In this situation, this would be the final result:
1-5
2-4
3-6

As you can see, lying about 1 and 3 got woman 4 her first choice (2), which she prefers to both 1 and 3. The switch improved her partner.